

Computer Science & IT

Data Structure & Programming



Stack

Lecture No. 01

By- Abhishek Sir



Recap of Previous Lecture



Topic

LTM - RMO

Topic

LTM - CMO

Topic

LTM - RMO, CMO

Topic

3-D array C-language

Topic

Topics to be Covered



Topic

stack as LIFO

Topic

Application:- permutation generator

Topic

Topic

Topic



Topic : Stack as LIFO



- 1 Stack is Linear one ended data structure
- 2 Stack works in principle of Last In first out
- 3 Stack maintain an Index top for insert and delete operation.



Stack



Empty stack: $top = -1$ (C-Implementation)





Stack



push(1) !

$$\text{top} = -1 + 1 = 0$$

$$\text{arr}[\text{top}] = 1;$$



Stack



push(2) : $top = 0 + 1 = 1$
 $arr[1] = 2$

Increment the top and
Insert new value in top Index of
array



Stack

if stack is full, Insertion results
in



Stack overflow



Stack



pop returns a value

$data = arr[top];$

$top = top - 1 = 1$

return data;



Stack



pop

$data = arr[top]$

$top = 0 - 1 = -1$

return data

first perform the deletion then
decrement the top.



Stack



pop from empty stack

stack Underflow





Topic : Stack Implementation

Array

```
#define Max 100
```

```
int arr[Max];
```

```
int top = -1;
```

if stack is empty

IsEmpty returns 1 (true)

else 0 (false)

```
int IsEmpty() {  
    if (top == -1)  
        return 1;  
    else  
        return 0;  
}
```



Topic : Stack Implementation

Array

```
#define Max 100
```

```
int arr[Max];
```

```
int top = -1;
```

```
void push (int data) {
```

```
    if (top == Max - 1) {
```

```
        printf ("Stack is full");
```

```
        return;
```

```
    }
```

```
    top = top + 1;
```

```
    arr[top] = data;
```

```
}
```



Topic : Stack Implementation



Array

```
#define Max 100
```

```
int arr[Max];
```

```
int top = -1;
```

```
int pop() {  
    int data;
```

```
    if (IsEmpty()) {  
        printf("stack is empty");
```

```
        return -1;
```

```
    }
```

```
    data = arr[top];
```

```
    top = top - 1;
```

```
    return data;
```

Limit.h
INT_MIN



Question

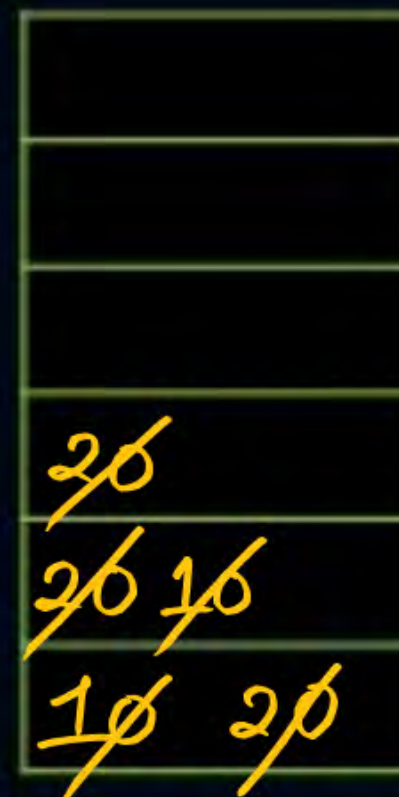


#Q The following sequence of operations is performed on a stack:

PUSH (10), PUSH (20), POP , PUSH (10), PUSH (20), POP, POP, POP, PUSH (20), POP

The sequence of values popped out is:

- (A) 20, 10, 20, 10, 20
- (B) 20, 20, 10, 10, 20
- (C) 10, 20, 20, 10, 20
- (D) 20, 20, 10, 20, 10 ✓



20, 20, 10, 10, 20



Question



#Q Consider the usual algorithm for determining whether a sequence of parentheses is balanced. What is the maximum number of parentheses that will appear on the stack AT ANY ONE TIME when the algorithm analyzes: $(()()())$?

(A) 1

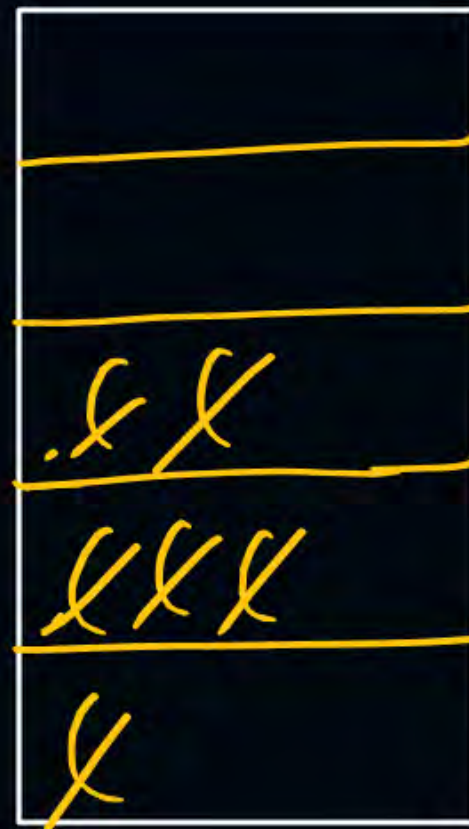
(B) 2

☒ (C) 3

(D) 4

$$S \rightarrow SS$$

$$S \rightarrow (S) \mid ()$$



(- push
) - pop
if stack is
empty then
balanced



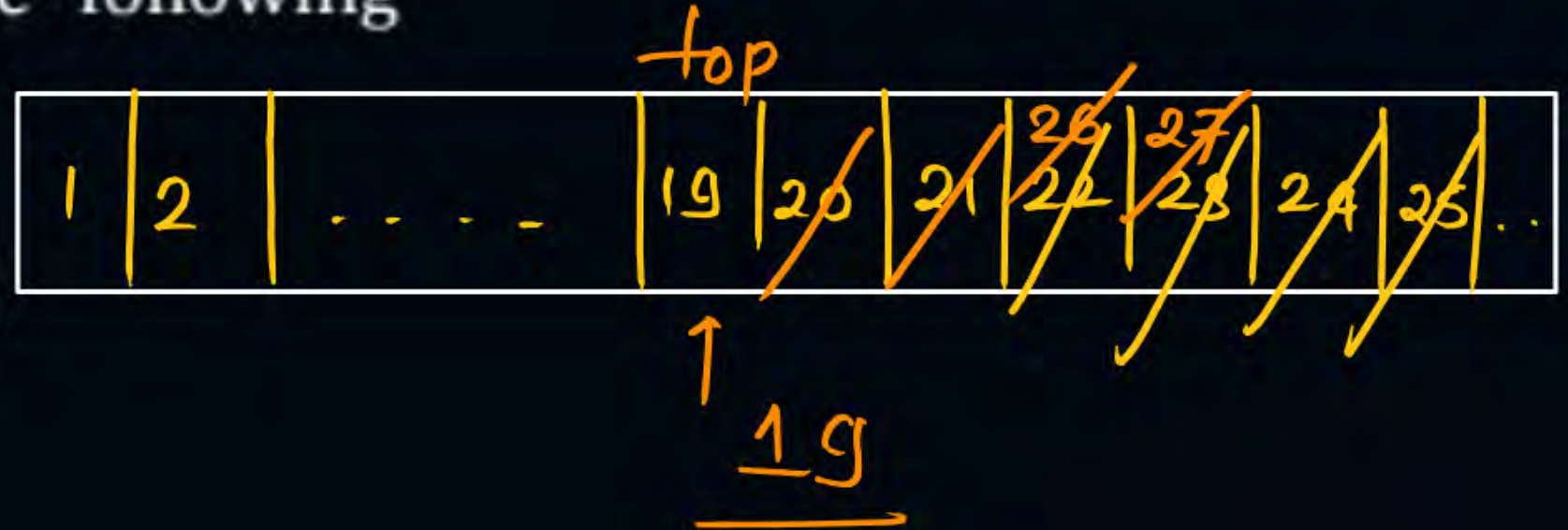
1. Push(1), Push(2).....Push(25)

II. Pop() , pop(). Pop(), Pop()

III. Push (26), Push(27) ✓

IV. Pop() , pop(). Pop(), Pop()

The Top element of the stack is 19





Question



#Q consider the array implementation of the using an array of size 100. Base Address is 100 and size is 2B Suppose the following operation is being performed:

I. Push(1), Push(2).....Push(50)

II. Pop() , pop(), Pop(), Pop(), Pop(), Pop()

III. Push (51),

The address of top element of the stack is 188.



0 1

44

$$100 + (44 - 0) \times 2$$

188

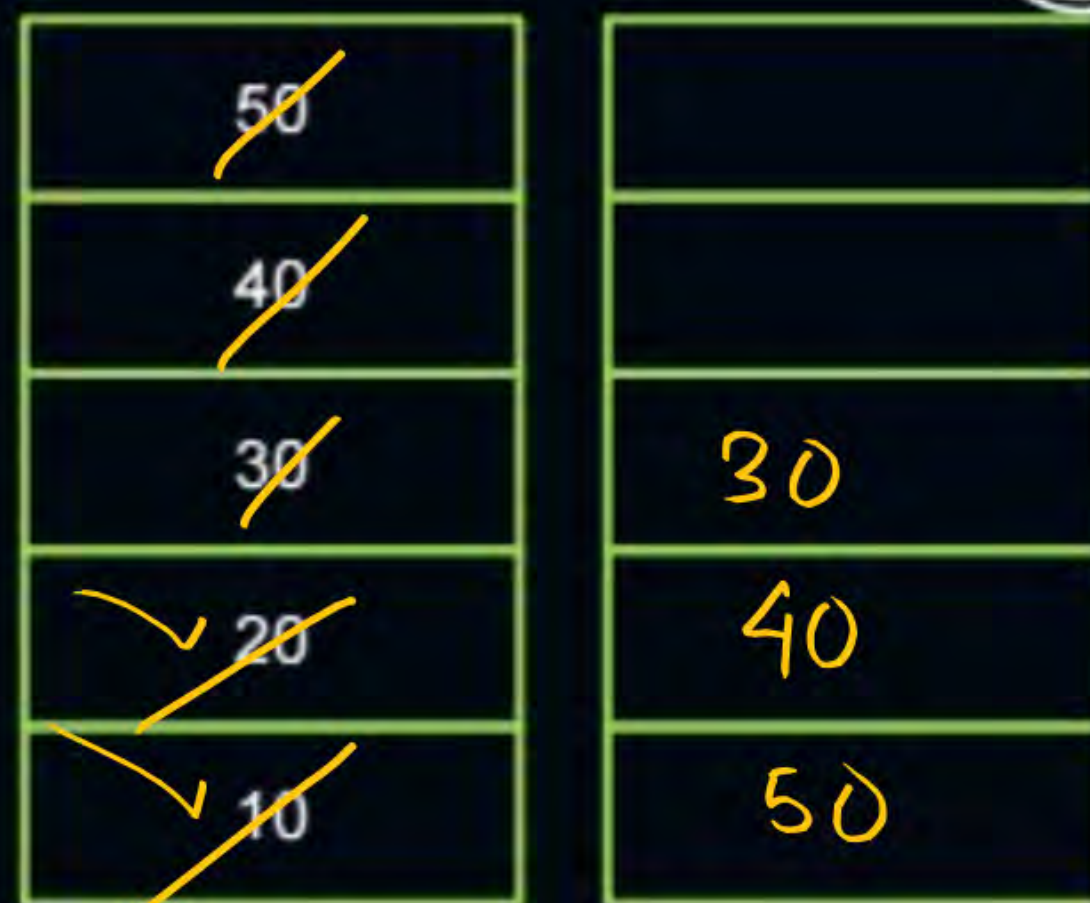


Question



#Q consider 2 stack S1 and s2 , stack s1 consists of 5 elements 10, 20, 30 40, 50 . 50 is on top of stack and s2 is empty . We want pop 10 and 20 and remaining element will be present in the same order. We can use S2 stack for temporary hold the value. Minimum number of push and pop required to achieve this is 14

pop pop
pop(30) push
pop(40) push
pop(50) push



S1

S2

pop(50) push
pop(40) push
pop(30) push

Question MSQ

#Q consider 2 stack S1 and s2 , stack s1 consists of 5 elements 10, 20, 30 40, 50 . 50 is on top of stack and s2 is empty . We can use S2 for temporarily holding values. Which of the following sequence of output is possible if Not allowed: Push from S2 $\rightarrow$ S1

50	10
40	20
30	30
20	40
10	50
S1	S2

(A) 10 20 30 40 50 ✓ *pop from s1 push s2 pop from s2*

(B) 10 20 50 40 30 ✗

(C) 20 10 50 40 30 ✗

(D) 40 30 20 50 10

Question MSQ

#Q consider 2 stack S1 and s2 , stack s1 consists of 5 elements 10, 20, 30 40, 50 . 50 is on top of stack and s2 is empty . We can use S2 for temporarily holding values. Which of the following sequence of output is possible if Not allowed: Push from S2 $\rightarrow$ S1



- (A) 10 20 30 40 50 ✓ *pop from s1 push s2 pop from s2*
- (B) 10 20 50 40 30 ✗
- (C) 20 10 50 40 30 ✗
- (D) 40 30 20 50 10 ✓

			...	77	78		105
--	--	--	-----	----	----	--	-----

-5

No. of element crossed
 $(78 - (-5)) = \underline{83}$

Size of Row/column: 1

$$105 - (-5) + 1 = \textcircled{111}$$



Generating Permutation Using Stack

We can generate as many permutations as possible of the string by pushing the characters in the some order onto a stack, but it may pop off the top character at any time.

Question

How many permutations are possible with a, b, c

a b c	push(a) pop push(b) pop push(c) pop ✓			
a c b	push(a) pop push(b) push(c) <table border="1"><tr><td>c</td></tr><tr><td>b</td></tr></table> pop pop ✓	c	b	
c				
b				
b a c	push(a) push(b) <table border="1"><tr><td>b</td></tr><tr><td>a</td></tr></table> pop pop push(c) pop ✓	b	a	
b				
a				
b c a	push(a) push(b) pop <table border="1"><tr><td>a</td></tr></table> push(c) pop pop	a		
a				
c a b	Not possible			
c b a	push(a) push(b) push(c) <table border="1"><tr><td>c</td></tr><tr><td>b</td></tr><tr><td>a</td></tr></table> pop pop pop	c	b	a
c				
b				
a				

Order of
element in fixed

c
b
a

With 3 elements 5 permutation are possible

Question

A program attempts to generate as many permutations as possible of the string 'abcd' by pushing the characters a, b, c, d in the same order onto a stack, but it may pop off the top character at any time. Which one of the following strings

CANNOT be generated using this program?

(a) abcd ✓

(b) dcba ✓

(c) cbad ✓

(d) cabd ✓

abcd

push(a) pop push(b) pop

push(c) pop push(d) pop

push(a,b,c,d) pop

push(a,b,c)

c
b
a

$n = 4$.

14 permutations are possible

d a b c ¹	c b a d ³	b a c d ⁵	a b c d ⁵
d a c b	c b d a	b a d c	a b d c
d b a c	c a b d	b c a d	a c b d
d b c a	c a d b	b c d a	a c d b
d c a b	c d b a	b d c a	a d b c
d c b a	c d a b	b d a c	a d c b

Question

How many permutations are possible with a, b, c, d

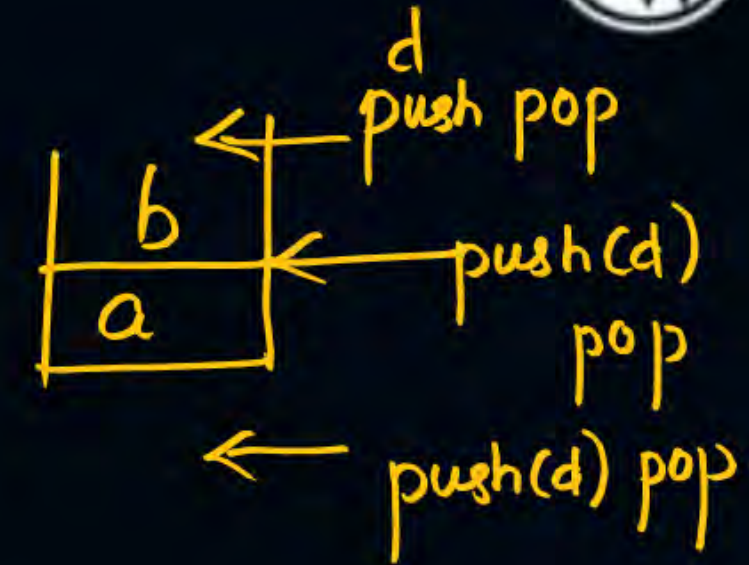
d a b c	
d a c b	
d b a c	
d b c a	
d c a b	
d c b a	✓

d
c
b
a

Question

How many permutations are possible with a, b, c, d

c b a d	push(a) push(b) push(c) pop ✓
c b d a	✓
c a b d	X Order is a followed by b
c a d b	X a followed by b
c d b a	✓
c d a b	X a followed by b



Question



How many permutations are possible with a, b, c, d

b a c d	push(a) push(b) pop pop push(c) pop push(d) pop
b a d c	push(a) push(b) pop pop push(c) push(d) pop pop
b c a d	push(a) push(b) pop push(c) pop pop push(d) pop
b c d a	push(a) push(b) pop push(c) pop push(d) pop pop
b d c a	push(a) push(b) pop push(c) push(d) pop pop pop
b d a c	Not possible

Question

How many permutations are possible with a, b, c, d

a b c d	push(a) pop ✓
a b d c	push(a) pop ✓
a c b d	push(a) pop ✓
a c d b	push(a) pop ✓
a d b c	push(a) pop Not possible
a d c b	pop(a) pop ✓



Topic : Stack as Permutation Generator

No. of permutation possible is

$$C_n = \frac{1}{n+1} 2^n C_n$$

$$C_1 = 1$$

$$C_5 = 42$$

$$C_2 = 2$$

$$C_6 = 136$$

$$C_3 = 5$$

$$C_4 = 14$$

Matrix chain multiplication

No. of ways 4 Matrices

Can be parenthesis

$$A_1 ((A_2 A_3) A_4)$$

$$A_1 (A_2 (A_3 A_4))$$

$$(A_1 A_2) (A_3 A_4)$$

$$((A_1 A_2) A_3) A_4$$

$$(A_1 (A_2 A_3)) A_4$$



Topic : GATE 1994

Which of the following permutations can be obtained in the output (in the same order) using a stack assuming that the input is the sequence 1, 2, 3, 4, 5 in that order?

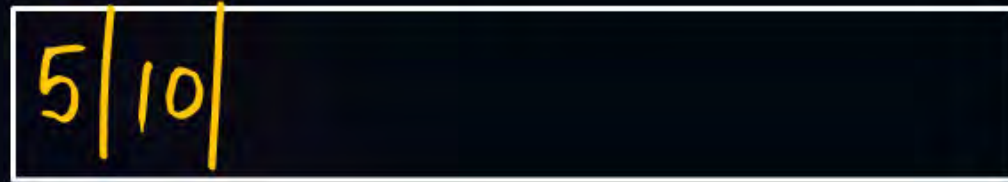
- ~~A.~~ 3, 4, 5, 1, 2 $\rightsquigarrow$

3
2
1

push(1) push(2) push(3) pop push(4) pop push(5) pop, pop pop
- B. 3, 4, 5, 2, 1
- C. 1, 5, 2, 3, 4 $\rightsquigarrow$ 1, 5, 4, 3, 2 correct order
- D. 5, 4, 3, 1, 2 $\rightsquigarrow$ 5, 4, 3, 2, 1

```
#include <stdio.h>
int *A, stkTop;
int stkFunc (int opcode, int val) {
    static int size=0, stkTop=0;
    switch (opcode) {
        case -1: size = val; break;
        case 0: if (stkTop < size) A[stkTop++] = val; break;
        default: if (stkTop) return A[--stkTop];
    }
    return -1;
}
```

stkTop = 2 C-program PYQ



```
int main() {
    int B[20]; A=B; stkTop = -1;
    stkFunc (-1, 10);
    stkFunc (0, 5);
    stkFunc (0, 10);
    printf ("%d\n", stkFunc(1, 0)+ stkFunc(1, 0));
}
```

The output of the program is 15

THANK - YOU